Document Number: P1020R0
Date: 2018-04-08
Project: Programming Language C++
Audience: Library Evolution Working Group
Author: Glen Joseph Fernandes (glenjofe@gmail.com), Peter Dimov (pdimov@pdimov.com)

Smart pointer creation with default initialization

This paper proposes adding the smart pointer creation functions `allocate_shared_default_init`, `make_shared_default_init`, and `make_unique_default_init` that perform default initialization.

Motivation

It is not uncommon for arrays of built-in types such as `unsigned char` or `double` to be immediately initialized by the user in their entirety after allocation. In these cases, the value initialization performed by `allocate_shared`, `make_shared`, and `make_unique` is redundant and hurts performance, and a way to choose default initialization is needed.

Implementation

The Boost Smart Pointers library has provided an implementation of this proposal since Boost 1.56 released in 2014. The Boost functions are named `allocate_shared_noinit`, `make_shared_noinit`, and `make_unique_noinit`, and are in widespread use.

Proposed Wording

All changes are relative to N4741.

Insert into 23.10.2 [memory.syn] as follows:

```
template<class T, class... Args> unique_ptr<T> make_unique(Args&&... args); //T is
not array
template<class T> unique_ptr<T> make_unique(size_t n); //T is U[]
template<class T, class... Args> unspecified make_unique(Args&&...) = delete; //T
is U[N]

template<class T> unique_ptr<T> make_unique_default_init(); //T is not array
template<class T> unique_ptr<T> make_unique_default_init(size_t n); //T is U[]
template<class T, class... Args> unspecified make_unique_default_init(Args&&...) =
delete; //T is U[N]
[...]
template<class T> shared_ptr<T> make_shared(const remove_extent_t<T>& u); //T is
U[N]
template<class T> shared_ptr<T> allocate_shared(const A& a, const
remove_extent_t<T>& u); //T is U[N]

template<class T> shared_ptr<T> make_shared_default_init(); //T is not U[]
template<class T> shared_ptr<T> allocate_shared_default_init(const A& a); //T is
not U[]
```

```
template<class T> shared_ptr<T> make_shared_default_init(size_t N); //T is U[]
template<class T> shared_ptr<T> allocate_shared_default_init(const A& a, size_t
N); //T is U[]
```

Insert after 23.11.1.4 [unique.ptr.create] p5 as follows:

```
template<class T> unique_ptr<T> make_unique_default_init(); //T is not array
```

Remarks: This function shall not participate in overload resolution unless `T` is not an array.

Returns: `unique_ptr<T>(new T)`.

```
template<class T> unique_ptr<T> make_unique_default_init(size_t n); //T is U[]
```

Remarks: This function shall not participate in overload resolution unless `T` is an array of unknown bound.

Returns: `unique_ptr<T>(new remove_extent_t<T>[n])`.

```
template<class T, class... Args> unspecified make_unique_default_init(Args&&...) =
delete; //T is U[N]
```

Remarks: This function shall not participate in overload resolution unless `T` is an array of known bound.

Change 23.11.3.6 [util.smartptr.shared.create] p1 as follows:

The common requirements that apply to all `make_shared`, ~~and~~ `allocate_shared`, `make_shared_default_init`, and `allocate_shared_default_init` overloads, unless specified otherwise, are described below.

```
template<class T, ...> shared_ptr<T> make_shared(args);
template<class T, class A, ...> shared_ptr<T> allocate_shared(const A& a, args);
template<class T, ...> shared_ptr<T> make_shared_default_init(args);
template<class T, class A, ...> shared_ptr<T> allocate_shared_default_init(const
A& a, args);
```

Change 23.11.3.6 [util.smartptr.shared.create] p3 as follows:

Effects: Allocates memory for an object of type `T` (or `U[N]` when `T` is `U[]`, where `N` is determined from `args` as specified by the concrete overload). The object is initialized from *`args`* as specified by the concrete overload. The `allocate_shared` and `allocate_shared_default_init` templates use a copy of `a` (rebound for an unspecified `value_type`) to allocate memory. If an exception is thrown, the functions have no effect.

Insert after 23.11.3.6 [util.smartptr.shared.create] p7.7 as follows:

— When a (sub)object of non-array type `U` is specified to have *no initial value*, `make_shared_default_init` and `allocate_shared_default_init` shall initialize this (sub)object via the expression `::new(pv) U`, where `pv` has type `void*` and points to storage suitable to hold an object of type `U`.

Insert after 23.11.3.6 [util.smartptr.shared.create] p22 as follows:

```
template<class T> shared_ptr<T> make_shared_default_init(); //T is not U[]
template<class T, class A> shared_ptr<T> allocate_shared_default_init(const A& a);
```

```
//T is not U[]
```

Returns: A `shared_ptr` to an object of type `T`, where each array element has *no initial value*.

Remarks: These overloads shall only participate in overload resolution when `T` is not of the form `U[]`.

[*Example:*

```
struct X { double data[1024]; };
shared_ptr<X> p = make_shared_default_init<X>();
//shared_ptr to a default-initialized X, with X::data left uninitialized

shared_ptr<double[1024]> q = make_shared_default_init<double[1024]>();
//shared_ptr to a default-initialized double[1024], with the elements left
uninitialized
```

—*end example*]

```
template<class T> shared_ptr<T> make_shared_default_init(size_t N); //T is U[]
template<class T, class A> shared_ptr<T> allocate_shared_default_init(const A& a,
size_t N); //T is U[]
```

Returns: A `shared_ptr` to an object of type `U[N]`, where `U` is `remove_extent_t<T>` and each array element has *no initial value*.

Remarks: These overloads shall only participate in overload resolution when `T` is of the form `U[]`.

[*Example:*

```
shared_ptr<double[]> p = make_shared_default_init<double[]>(1024);
//shared_ptr to a default-initialized double[1024], with the elements left
uninitialized
```

—*end example*]

References

- N4741, Working Draft, Standard for Programming Language C++, http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4741.pdf